形式语言与自动机：第一次 Python 编程作业

组长姓名：_____

组长学号：_____

签署日期：_____年_____月_____日

小组成员与分工

姓名	负责内容
黄爱心	第一题：语言串接
廖敏佳	第二题：有界星操作
刘倪羽晗	第三题：DFA 转移与模拟
刘嘉瑜	第四题：NFA 子集构造核心算法
刘雅静（组长）	第四题：转移表输出与验证、整体整合

运行环境与方法

使用 Python 3，仅依赖标准库。在项目根目录依次执行：

```
python homework1/question1.py
python homework1/question2.py
python homework1/question3.py
python homework1/question4.py
```

下文包含各题的输入、完整源码及结果。集合中的元素不计顺序，第一题的实际打印顺序可能与本文不同。

第一题：语言的串接

自行构造的语言

- `L1 = {"a", "AB", "cde", "FGHI"}`，包含 4 个字符串。
- `L2 = {"10", "211", "3222", "43333"}`，包含 4 个字符串。

依次拼接每个 `L1` 元素和每个 `L2` 元素，以集合保存结果并去重。

完整源码：question1.py

```
#拼接函数定义
def concatenate_language(L1,L2):
    result_language=set()
    for x in L1:
        for y in L2:
            result_language.add(x+y)
    return result_language
#拼接
if __name__=='__main__':
    #自行构造L1，L2
    L1={"a","AB","cde","FGHI"}
    L2={"10","211","3222","43333"}
    print("语言L1=",L1)
    print("语言L2=",L2)
    L3=concatenate_language(L1,L2)
    print("串接后的语言 L1L2 中的所有字符串如下：")
    for string in L3:
        print(string)
```

运行结果

共得到 16 个不同的字符串。以下按 `L1` 的元素分组整理，程序实际逐行打印全部结果。

L1 中的字符串	与 L2 串接得到的字符串
a	a10 、 a211 、 a3222 、 a43333
AB	AB10 、 AB211 、 AB3222 、 AB43333
cde	cde10 、 cde211 、 cde3222 、 cde43333
FGHI	FGHI10 、 FGHI211 、 FGHI3222 、 FGHI43333

第二题：语言的有界星操作

自行构造的语言与参数

`L = {"q", "qu", "u"}` ，包含 3 个字符串； `T = 3` 。

使用深度优先搜索生成最多拼接 3 个语言元素的字符串，以集合去重。拼接 0 次得到空串，输出时用 `ε` 表示。这里限制的是拼接的语言元素个数，不是结果的字符数。

完整源码：question2.py

```
def bounded_star(L, T):
    """
    计算有界星操作S^T(L)
    L: 语言，字符串集合
    T: 上界，最多拼接T个L中的字符串
    """
    if not isinstance(L, (set, frozenset)):
        raise TypeError("L 必须是字符串集合（set 或 frozenset）")
    if any(not isinstance(w, str) for w in L):
        raise TypeError("L 中的每个元素都必须是字符串")
    if isinstance(T, bool) or not isinstance(T, int):
        raise TypeError("T 必须是非负整数，不能是布尔值")
    if T < 0:
        raise ValueError("T 不能为负数")

    result = set()

    def dfs(current, depth):
        result.add(current)
        if depth == T:
            return
        for w in L:
            dfs(current + w, depth + 1)

    dfs("", 0)
    return result


if __name__ == "__main__":
    # |L| >= 2
    L = {"q", "qu", "u"}

    T = 3
    result = bounded_star(L, T)

    print(f"L = {L}")
    print(f"T = {T}")
    print(f"S^{T}(L) 中共有 {len(result)} 个字符串：")
    for s in sorted(result, key=lambda x: (len(x), x)):
        if s == "":
            print("ε")   # 空串用ε表示
        else:
            print(s)
```

运行结果

`T = 3`，共得到 33 个不同的字符串（含空串）。以下按字符长度分组列出全部结果；程序按长度、字典序逐行打印。

字符长度	字符串	数量
0	ε （空串）	1
1	q 、 u	2
2	qq 、 qu 、 uq 、 uu	4
3	qqq 、 qqu 、 quq 、 quu 、 uqq 、 uqu 、 uuq 、 uuu	8
4	qqqu 、 qquq 、 qquu 、 quqq 、 ququ 、 quuq 、 quuu 、 uqqu 、 uquq 、 uquu 、 uuqu	11
5	qququ 、 quqqu 、 ququq 、 ququu 、 quuqu 、 uququ	6
6	quququ	1

例如， qu 既可由一个元素 qu 得到，也可由两个元素 q 与 u 拼接得到，因此最终结果需要去重。

第三题：DFA 的转移函数与扩展转移函数

自动机与自行构造的用例

初态为 `A`，终态集合为 `{A, B}`，字母表为 `{0, 1}`。

当前状态	输入 0	输入 1
A	A	B
B	A	C
C	C	C

正例为 `101`，负例为 `111`，长度均为 3。`Edelta(q, s)` 逐字符调用 `delta(q, a)`，打印每次转移并返回最终状态。

完整源码：question3.py

```python
TRANSITIONS = {
    "A": {"0": "A", "1": "B"},
    "B": {"0": "A", "1": "C"},
    "C": {"0": "C", "1": "C"},
}

START_STATE = "A"
ACCEPT_STATES = {"A", "B"}


def delta(q, a):
    """Return the state reached from q after reading a."""
    if q not in TRANSITIONS:
        raise ValueError(f"Unknown state: {q}")
    if a not in TRANSITIONS[q]:
        raise ValueError(f"Character is not in the input alphabet: {a}")
    return TRANSITIONS[q][a]


def Edelta(q, s):
    """Print each transition while reading s, then return the final state."""
    if q not in TRANSITIONS:
        raise ValueError(f"Unknown state: {q}")

    current_state = q
    for index, character in enumerate(s, start=1):
        next_state = delta(current_state, character)
        print(
            f"Step {index}: delta({current_state}, {character}) = {next_state}"
        )
        current_state = next_state
    return current_state


def run_case(label, input_string):
    print(f"{label}, input string: {input_string}")
    final_state = Edelta(START_STATE, input_string)
    result = "accepted" if final_state in ACCEPT_STATES else "rejected"
    print(f"Final state: {final_state}, result: {result}")
    print()


if __name__ == "__main__":
    run_case("Positive example", "101")
    run_case("Negative example", "111")
```

运行结果

```
Positive example, input string: 101
Step 1: delta(A, 1) = B
Step 2: delta(B, 0) = A
Step 3: delta(A, 1) = B
Final state: B, result: accepted

Negative example, input string: 111
Step 1: delta(A, 1) = B
Step 2: delta(B, 1) = C
Step 3: delta(C, 1) = C
Final state: C, result: rejected
```

正例不含连续两个 `1`，最终到达终态 `B`；负例含有连续两个 `1`，最终到达非终态 `C`。

第四题：NFA 的子集构造算法

输入与算法

使用题目指定的 NFA，转移表见下方源码中的 `NFA` 。初态为 `1` ，终态集合为 `{9}` ，字母表为 `{r, b}` ，没有空串转移。

以 `{1}` 为 DFA 初态，对每个已发现的子集和每个输入字符，求子集中所有 NFA 状态的目标状态并集。新子集加入待处理队列，直到所有可达子集处理完毕。包含 NFA 终态 `9` 的子集为 DFA 终态。

完整源码：question4.py

```python
"""第四题：使用子集构造法将题目中的 NFA 转换为 DFA。"""

from collections import deque


# 转移表中每个值都是状态集合；本题没有 epsilon 转移。
NFA = {
    1: {"r": {2, 4}, "b": {5}},
    2: {"r": {4, 6}, "b": {1, 3, 5}},
    3: {"r": {2, 6}, "b": {5}},
    4: {"r": {2, 8}, "b": {1, 5, 7}},
    5: {"r": {2, 4, 6, 8}, "b": {1, 3, 7, 9}},
    6: {"r": {2, 8}, "b": {3, 5, 9}},
    7: {"r": {4, 8}, "b": {5}},
    8: {"r": {4, 6}, "b": {5, 7, 9}},
    9: {"r": {6, 8}, "b": {5}},
}
ALPHABET = ("r", "b")
START = 1
FINAL = {9}


def subset_construction(nfa, alphabet, start, final):
    """返回可达 DFA 状态、转移表、初态和终态集合。

    DFA 的一个状态是 NFA 状态的子集，使用 frozenset 作为字典键。
    对子集 S 和字符 a，delta_D(S, a) = 并集{delta_N(q, a): q 属于 S}。
    """
    dfa_start = frozenset({start})
    states = [dfa_start]
    discovered = {dfa_start}
    pending = deque([dfa_start])
    transitions = {}

    while pending:
        current = pending.popleft()
        transitions[current] = {}
        for symbol in alphabet:
            target = set()
            for state in current:
                target.update(nfa[state][symbol])
            target = frozenset(target)
            transitions[current][symbol] = target

            if target not in discovered:
                discovered.add(target)
                states.append(target)
                pending.append(target)

    # 子集中只要包含一个 NFA 终态，该子集就是 DFA 终态。
```

```
    dfa_final = {state for state in states if state & final}
    return states, transitions, dfa_start, dfa_final


def format_subset(states):
    return "{" + ",".join(str(state) for state in sorted(states)) + "}"


def print_dfa(states, transitions, start, final, alphabet):
    names = {state: f"D{i}" for i, state in enumerate(states)}
    print("DFA 转移表（-> 表示初态，* 表示终态）")
    print(f"{'标记':<6}{'状态':<8}{'NFA 状态子集':<24}" +
          "".join(f"{symbol:<8}" for symbol in alphabet))
    print("-" * 60)
    for state in states:
        mark = ("->" if state == start else "") + ("*" if state in final else "")
        targets = "".join(f"{names[transitions[state][a]]:<8}" for a in alphabet)
        print(f"{mark:<6}{names[state]:<8}{format_subset(state):<24}{targets}")
    print(f"\n状态数：{len(states)}")
    print("初态：" + names[start])
    print("终态：" + ", ".join(names[state] for state in states if state in final))


if __name__ == "__main__":
    dfa_states, dfa_delta, dfa_start, dfa_final = subset_construction(
        NFA, ALPHABET, START, FINAL
    )
    print_dfa(dfa_states, dfa_delta, dfa_start, dfa_final, ALPHABET)
```

运行结果

以下将程序打印的转移表排版为表格。`->` 表示初态，`*` 表示终态。

标记	DFA 状态	NFA 状态子集	r	b
->	D0	{1}	D1	D2
	D1	{2,4}	D3	D4
	D2	{5}	D3	D5
	D3	{2,4,6,8}	D3	D6
	D4	{1,3,5,7}	D3	D6
*	D5	{1,3,7,9}	D3	D2
*	D6	{1,3,5,7,9}	D3	D6

状态数为 **7**，初态为 **D0**，终态集合为 **{D5, D6}**。

例如，子集 `{2,4}` 读入 `r` 时，目标子集为 `{4,6} ∪ {2,8} = {2,4,6,8}`，因此表中 `D1` 在 `r` 下转移到 `D3`。`D5` 与 `D6` 的子集均包含 `9`，因此标为终态。